



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

OPERATING SYSTEMS AND SYSTEMS PROGRAMMING (25CS2104E)

2026-27, Term-I

Project Problem Statement Submission Form

Section / Team Details

Section No.: 10

Team No.: 16

Project Title: SYSTEM CALLS

Team Members

Roll Number	Student Name	Signature
2520090111	G. SATWIK	<i>Satwika.</i>
2520090198	S. SIRI CHANDANA	<i>Siri</i>

I. Abstract

System calls provide the essential interface between user applications and the Linux operating system, allowing programs to request services from the kernel in a controlled and secure manner. This project focuses on understanding and demonstrating the role of system calls in process management, file handling, memory management, and communication between processes. The problem addressed is how user-level programs can safely access operating system resources without directly interacting with hardware or kernel components.

The project explores important Operating Systems and System Programming concepts such as process creation and termination, `fork()`, `exec()`, `wait()`, process identification, file operations, and inter-process communication. A Linux-based solution will be developed using C programs that invoke system calls directly. The implementation will demonstrate how a parent process creates a child process using `fork()`, how the child executes another program using `exec()`, and how the parent waits for the child using `wait()`. Additional demonstrations may include system calls such as `open()`, `read()`, `write()`, and `close()` for file management.

The expected outcome is a practical understanding of how system calls act as a bridge between user programs and the Linux kernel. The project will help students observe process creation, program execution, resource management, and kernel-level services through real Linux execution, thereby strengthening their understanding of Operating Systems and System Programming concepts.

for applications to request services from the operating system kernel. **System calls** provide this essential interface between user programs and the Linux kernel.

This project aims to study and practically demonstrate how system calls enable communication between user applications and the operating system. The project will address the problem of understanding how operations such as process creation, program execution, process synchronization, and file management are performed at the system level. Using Linux and C programming, system calls such as `fork()`, `exec()`, `wait()`, `open()`, `read()`, `write()`, and `close()` will be implemented and analyzed.

The project is relevant to **Operating Systems and Systems Programming** because system calls are fundamental to process management, resource management, file systems, and interaction with kernel services. The practical implementation will help demonstrate the transition from user mode to kernel mode and provide a clear understanding of how Linux manages and controls system resources securely and efficiently.

3. Objectives

- **To understand the concept of system calls** and their role as an interface between user programs and the Linux kernel.
- **To study different types of system calls**, including process management, file management, memory management, and communication-related system calls.
- **To implement process-related system calls** such as `fork()`, `exec()`, and `wait()` using C programming in a Linux environment.
- **To understand process creation and execution**, including parent-child process relationships and process IDs.
- **To analyze how user programs interact with kernel services** through system calls.

4. Proposed Methodology

The project will be designed and implemented in a **Linux environment using C programming** to demonstrate the working of system calls and their interaction with the operating system kernel. The methodology focuses on developing small practical programs that invoke different system calls and observe their execution and results.

The project will follow these major steps:

1. **Linux Environment Setup:** Set up a Linux environment using Ubuntu or WSL and install the required C compiler and development tools.
2. **Study of System Calls:** Identify and study commonly used system calls related to process management and file management, such as `fork()`, `exec()`, `wait()`, `open()`, `read()`, `write()`, and `close()`.
3. **Process Management Implementation:** Develop a C program using `fork()` to create a child process, `exec()` to execute another program, and `wait()` to synchronize the parent and child processes. Process IDs will be displayed to observe the parent-child relationship.
4. **Execution and Testing:** Compile and execute the programs in Linux and observe outputs such as process IDs, execution status, file contents, and return values.
5. **Analysis:** Analyze how user programs request services from the Linux kernel through system calls and document the results.

Overall Flow:

User Program → System Call → Linux Kernel → Resource/Service → Return Result → User Program

4. **Execution and Testing:** Compile and execute the programs in Linux and observe outputs such as process IDs, execution status, file contents, and return values.
5. **Analysis:** Analyze how user programs request services from the Linux kernel through system calls and document the results.

Overall Flow:

User Program → System Call → Linux Kernel → Resource/Service → Return Result → User Program

5. Operating Systems Concepts / Linux APIs Used

(List the relevant Operating Systems concepts, Linux system calls, POSIX APIs, or mechanisms that will be used.)

OS Concept / Linux API / System Call	Purpose in the Project
System Calls	Provides an interface between user programs and the Linux kernel to request OS services.
Process Management	Used to understand process creation, execution, termination, and synchronization.
fork()	Creates a new child process from the parent process.
exec()	Replaces the child process's program with another executable program.
wait() / waitpid()	Allows the parent process to wait for the child process to complete.
File Management	Demonstrates how Linux manages files through system calls.

6. Individual Contribution

(Clearly specify the responsibility of each team member in the project.)

Roll Number	Student Name	Individual Responsibility
2520090111	G .SATWIK	Study the concepts of system calls and process management. Implement and test fork(), exec(), wait(), getpid(), and getppid(). Analyze parent-child process creation and synchronization. Study Linux file management system calls
2520090198	V.SIRI CHANDANA	Set up the Linux environment, integrate and test all programs, analyze the overall results, prepare .Implement and test open(), read(), write(), and close(). Analyze file descriptors and document the execution results. project documentation, and present the project.

8. Expected Outcome

The project is expected to provide a practical understanding of how Linux system calls act as a bridge between user applications and the operating system kernel. Students will gain hands-on knowledge of process management, file management, process synchronization, file descriptors, and Linux system programming. The project will also demonstrate how operating systems provide controlled and secure access to system resources.

9. GitHub Repository

GitHub Repository Link: _____ https://github.com/satwikagandhe99-spec/16_OSSP-PROJECT

Repository Created: yes

☒ Yes ☐ No

Faculty Name and Signature

Faculty Name: Mr. Ragupathi

Signature: 

Remarks: _____

Faculty Use

Project Approved: _____

☒ Yes ☐ No

Date: 21/9/2026

Faculty Signature: 